

WINTERSEMESTER 2023/24

Studiengang/Abschluss: Wirtschaftsinformatik
Prüfungsfach: Programmierung
Prüfungsnummer: **5706**
Prüfer: Prof. Dr. Andreas Biesdorf
Anzahl der Arbeitsblätter: 10 Seiten (ohne Deckblätter)
Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:

Eine einseitig handbeschriebene DIN A4-Seite Python-Befehle oder -Syntax (z.B. die Syntax einer for- oder while-Schleife), jedoch keine Theorie oder Strukturen konkreter Implementierungen kompletter Aufgaben in Deutsch oder Englisch.

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

.....

Platznummer:

Punkte: _____ von 90 möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtsführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Die Klausur besteht aus 7 Aufgaben zu den in der Lehrveranstaltung behandelten Themenbereichen. Die volle Punktzahl der Klausur beträgt 90 Punkte. Die Punktzahl ist bei jeder Aufgabe mit angegeben.

Aufgabe	Mögliche Punkte	Erreichte Punkte
A1	20	
A2	15	
A3	10	
A4	10	
A5	15	
A6	15	
A7	5	
Evtl. Bonus		
Summe	90	

Kurzreferat gehalten ☐ Nein ☐ Ja, zum Thema: _____

Bitte dokumentieren Sie die Antworten zu allen Aufgaben in der entsprechenden Textbox auf dem Blatt oder der entsprechend benannten Datei (z.B. aufgabe1.txt oder aufgabe7.py) in dem Ordner auf der VM.

Aufgabe 1

- 1) Warum ist es zwingend erforderlich in größeren Projekten Methoden des Software-Engineerings zu verankern? Benennen Sie exemplarisch vier Gründe für den Einsatz von Software-Engineering. **[2 Punkte]**

- 2) Nennen und erläutern Sie jeweils drei Beispiele für Qualitätsmerkmale für Software aus Benutzer- und aus Entwicklersicht. **[3 Punkte]**

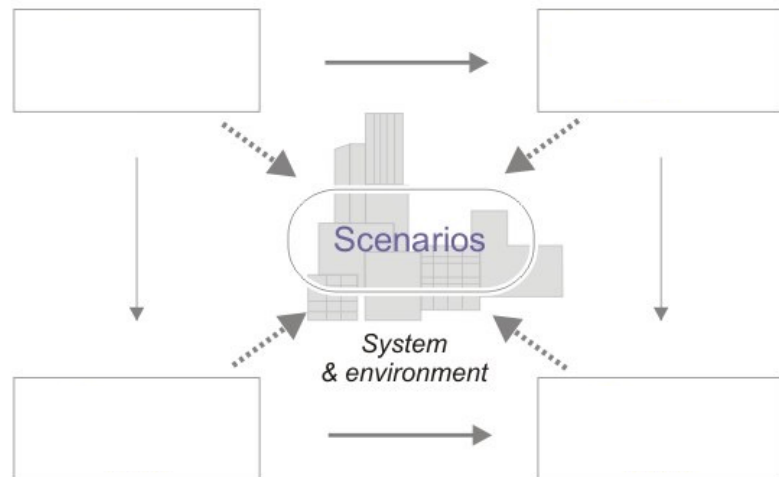
Drei Beispiele für Qualitätsmerkmale aus Benutzersicht:

Drei Beispiele für Qualitätsmerkmale aus Entwicklersicht:

- 3) Erläutern Sie den Unterschied zwischen dem Lasten- und Pflichtenheft. Welche Informationen finde ich in welchem der beiden Dokumente? **[5 Punkte]**

- 4) Was versteht man unter „Versteckten Anforderungen“? **[3 Punkte]**

- 5) Philippe Kruchten bildet Software-Architektur in 4+1 Sichten ab. Welche vier Sichten beinhaltet sein 4+1-Modell und welche Informationen werden in diesen Sichten jeweils abgebildet? Bitte ergänzen Sie die vier fehlenden Begriffe in der folgenden Grafik und erläutern die 4+1 verschiedenen Sichten stichwortartig (7 Punkte)



Aufgabe 2

- a) Gegeben ist eine Python-Funktion, die den Blutdruck einer Person in Abhängigkeit des systolischen und diastolischen Drucks klassifiziert:

prog_ss23/src/aufgabe02.py

```
def blutdruck_klassifizierung(systolisch, diastolisch):  
    if systolisch < 120 and diastolisch < 80:  
        return "Normal"  
    elif 120 <= systolisch < 130 and diastolisch < 80:  
        return "Erhöht"  
    elif 130 <= systolisch < 140 or 80 <= diastolisch < 90:  
        return "Bluthochdruck (Stufe 1)"  
    elif systolisch >= 140 or diastolisch >= 90:  
        return "Bluthochdruck (Stufe 2)"  
    else:  
        return "Hypertensiver Krisenzustand"
```

1. Benennen Sie fünf (möglichst überschneidungsfreie) Äquivalenzklassen zum Testen der Funktion. Erläutern Sie kurz, warum Sie diese Klassen gewählt haben. **[5 Punkte]**

2. Erstellen Sie in der Datei prog_ss23/test/test_aufgabe04.py Tests mithilfe von pytest, die die Funktion mit jeweils einem repräsentativen Wert aus jeder Äquivalenzklasse testet. **[2.5 Punkte]**

- b) Gegeben ist folgende iterative Python-Funktion, die eine Liste von Zahlen entgegennimmt und die Summe aller positiven Zahlen der Liste erstellt.

prog_ss23/src/aufgabe02.py

```
def finde_erste_negative_zahl(zahlenliste):  
    index = 0  
    while index < len(zahlenliste):  
        if zahlenliste[index] < 0:  
            return zahlenliste[index]  
        index += 1  
    return "Keine negative Zahl gefunden"
```

1. Benennen Sie fünf (möglichst überschneidungsfreie) Äquivalenzklassen zum Testen der Funktion. Erläutern Sie kurz, warum Sie diese Klassen gewählt haben. **[5 Punkte]**

2. Erstellen Sie in der Datei prog_ss23/test/test_aufgabe04.py Tests mithilfe von pytest, die die Funktion mit jeweils einem repräsentativen Wert aus jeder Äquivalenzklasse testet. **[2.5 Punkte]**

Aufgabe 3

Gegeben sei die folgende – offensichtlich nicht sehr robust implementierte – Funktion namens `get_ratio_at_position`, die zwei Listen von Zahlen und einen Index als Parameter entgegennimmt. Die Funktion soll den Quotienten der Zahlen in den Listen am gegebenen Index zurückgeben. (10 Punkte)

`prog_ss23/src/aufgabe03.py`

```
def get_ratio_at_position(list_a, list_b, pos_in_list):
    """Assumes: list_a and list_b are lists of equal length of numbers
    Returns: List_a[pos_in_list]/List_b[pos_in_list]"""

    if (
        not isinstance(list_a, list)
        or not isinstance(list_b, list)
        or not isinstance(pos_in_list, int)
    ):
        raise TypeError

    return float(list_a[pos_in_list]) / float(list_b[pos_in_list])

# Beispielaufruf und Überprüfung
assert get_ratio_at_position([2, 3, 6, 9], [1, 2, 3, 4], 2) == 2

assert not get_ratio_at_position([2, 3, 6, 9], [1, 2, 3, 4], 3) == 3
```

Bitte überarbeiten Sie die **Funktion** `get_ratio_at_position` um folgendes Verhalten:

- Sofern die beiden ersten Parameter keine Liste oder der dritte Parameter kein Integer ist, wird aktuell intern eine Exception vom Typ **TypeError** erzeugt. Bitte fangen Sie die Exception ab und geben eine benutzerdefinierte Fehlermeldung aus.
- Ergänzen Sie die Funktion um die Behandlung von Exceptions vom Typ **IndexError**, wenn der angegebene Index außerhalb des Bereichs der Liste liegt.
- Ergänzen Sie die Funktion um die Behandlung von Exceptions vom Typ **ZeroDivisionError**, wenn eine Division durch Null erfolgt.
- Wie können Sie den Fehler abfangen, wenn die Listen nicht nur Zahlen enthalten, sondern z.B. einen String und dadurch die Division fehlschlägt?
- Nur wenn **keine Exception** auftritt, geben Sie den modifizierten String zurück.
- Beenden Sie das Programm immer mit einer Nachricht, die bestätigt, dass die Funktion `get_ratio_at_position` ausgeführt wurde, unabhängig davon, ob eine Exception aufgetreten ist oder nicht.

Aufgabe 4

Schreiben Sie ein Programm, das eine Textdatei liest, den Inhalt Zeile für Zeile mittels der Atbash-Chiffre verschlüsselt bzw. wieder entschlüsselt und das Ergebnis in eine Ausgabedatei schreibt. Die Atbash-Chiffre ist ein einfaches Verschlüsselungsverfahren, bei dem jeder Buchstabe des Alphabets durch den Buchstaben ersetzt wird, der im Alphabet an der symmetrischen Position steht (A wird zu Z, B wird zu Y, usw.). Alle nötigen Informationen und Befehle sollten über die Kommandozeile angegeben werden. **(10 Punkte)**

Hinweis: der Code für die Ver- und Entschlüsselung via Atbash-Chiffre ist schon in der Datei vorgegeben und muss nicht implementiert werden!

Beispielhafter Aufruf (bitte gehen Sie für den Aufruf in das src-Verzeichnis):

```
python3 aufgabe04.py -i <infile> -o <outfile> -c/-d
```

Argumente / Flags auf der Kommandozeile:

--infile/-i: gibt die Eingabedatei an

--outfile/-o: gibt die Ausgabedatei an

--chiffrieren/-c: Eingabedatei soll verschlüsselt werden und in Ausgabedatei gespeichert werden

--dechiffrieren/-d: Eingabedatei ist schon verschlüsselt und soll dechiffriert und dann in Ausgabedatei gespeichert werden

--help/-h: Ausgabe des Programmaufrufs

prog_ss23/src/aufgabe04.py

```
def atbash_chiffrieren(input_string):
    chiffrierter_text = ""

    for char in input_string:
        if char.isalpha():
            # Bestimmen der Position des Buchstabens im Alphabet
            ascii_offset = 65 if char.isupper() else 97
            chiffrierter_text += chr(25 - (ord(char) - ascii_offset) +
                                     ascii_offset)
        else:
            chiffrierter_text += char
    return chiffrierter_text

def atbash_dechiffrieren(input_string):
    # Bei der Atbash-Chiffre ist die Chiffrierung identisch mit der Dechiffrierung
    return atbash_chiffrieren(input_string)
```

Aufgabe 5

Gegeben ist das folgende Interface eines Stacks, bei der nur die `__init__` und die `push`-Methode implementiert ist:

prog_ss23/src/aufgabe05.py

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        """Entfernt und gibt das oberste Element des Stacks zurück."""
        raise NotImplementedError

    def peek(self):
        """Gibt das oberste Element des Stacks zurück, ohne es zu entfernen."""
        raise NotImplementedError

    def is_empty(self):
        """Gibt True zurück, wenn der Stack leer ist, sonst False."""
        raise NotImplementedError

    def display(self):
        """Zeigt alle Elemente im Stack an."""
        raise NotImplementedError
```

- a) Bitte implementieren Sie die vier Methoden `pop`, `peek`, `is_empty` und `display`. Bitte verwenden Sie hierfür intern in Ihrem Code eine Liste und keine „Double Ended Queue“ wie z.B. `collections.deque` **[8 Punkte]**
- b) Ergänzen Sie eine Funktion `get(i)`, welche Ihnen das `i`-te Element des Stacks zurückgibt. **[4 Punkte]**
- c) Ergänzen Sie eine Funktion `del(i)`, welche das `i`-te Element des Stacks entfernt **[3 Punkte]**

Aufgabe 6

Die Buchstaben eines beliebigen Strings sollen alphabetisch sortiert werden.

Gegeben sei hierfür folgender Rahmen des Codes:

prog_ss23/src/aufgabe06.py

```
def insertionsort(input_text):  
    # Ihre Implementierung hier  
    pass  
  
# Testen Sie Ihre Implementierung mit dem folgenden Code  
text = input("Bitte Text ohne Sonder- und Leerzeichen eingeben: ").lower()  
  
# Erwartete Ausgabe: identische Zeichen, jedoch alphabetisch sortiert  
print(insertionsort (text))
```

Bitte implementieren Sie die Funktion `insertionsort` unter Verwendung des Insertion-Sort-Verfahrens. **[15 Punkte]**

Aufgabe 9

Bitte geben Sie für die folgenden fünf Programmiersprachen jeweils in ca. einem Satz den typischen Verwendungszweck an. Ausführliche Erläuterungen sind nicht erforderlich!
[5 Punkte]

Ein Beispiel:

Visual Basic – Entwicklung von Windows-Anwendungen

Kotlin:

Erlang:

Swift:

Selenium:

Ruby:

Viel Erfolg bei der Bearbeitung der Aufgaben!